

Database management system (DBMS)

M6 - Lecture Notes

Concurrency Control

Dr. Rishikeshan C A
Associate Professor
SCOPE, VIT Chennai

Modified from

Database Design – 2nd Edition (Adrienne Watt)

(https://opentextbc.ca/dbdesign01/)

&

Database System Concepts, 7th Ed.

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use



Concurrency Control

- A database must provide a mechanism that will ensure that all possible schedules are
  - either conflict or view serializable, and
  - are recoverable and preferably cascadeless
- A policy in which only one transaction can execute at a time generates serial schedules, but provides a poor degree of concurrency
  - Are serial schedules recoverable/cascadeless?
- Testing a schedule for serializability *after* it has executed is a little too late!
- **Goal** – to develop concurrency control protocols that will assure serializability.

- Different concurrency control protocols provide different tradeoffs between the amount of concurrency they allow and the amount of overhead that they incur.

©Silberschatz, Korth and Sudarshan



Concurrency Control vs. Serializability Tests

- Concurrency-control protocols allow concurrent schedules, but ensure that the schedules are conflict/view serializable, and are recoverable and cascadeless .
- Concurrency control protocols generally do not examine the precedence graph as it is being created
  - Instead a protocol imposes a discipline that avoids nonseralizable schedules.





Lock-Based Protocols

- A lock is a mechanism to control concurrent access to a data item
- Data items can be locked in two modes :
  1. *exclusive (X) mode*. Data item can be both read as well as written. X-lock is requested using **lock-X** instruction.
  2. *shared (S) mode*. Data item can only be read. S-lock is requested using **lock-S** instruction.
- Lock requests are made to the concurrency-control manager by the programmer. Transaction can proceed only after request is granted.



Lock-Based Protocols (Cont.)

- **Lock-compatibility matrix**

	S	X
S	true	false
X	false	false

- A transaction may be granted a lock on an item if the requested lock is compatible with locks already held on the item by other transactions
- Any number of transactions can hold shared locks on an item,
  - But if any transaction holds an exclusive on the item no other transaction may hold any lock on the item.
- If a lock cannot be granted, the requesting transaction is made to wait till all incompatible locks held by other transactions have been released. The lock is then granted.

©Silberschatz, Korth and Sudarshan



Lock-Based Protocols (Cont.)

- Example of a transaction performing locking:

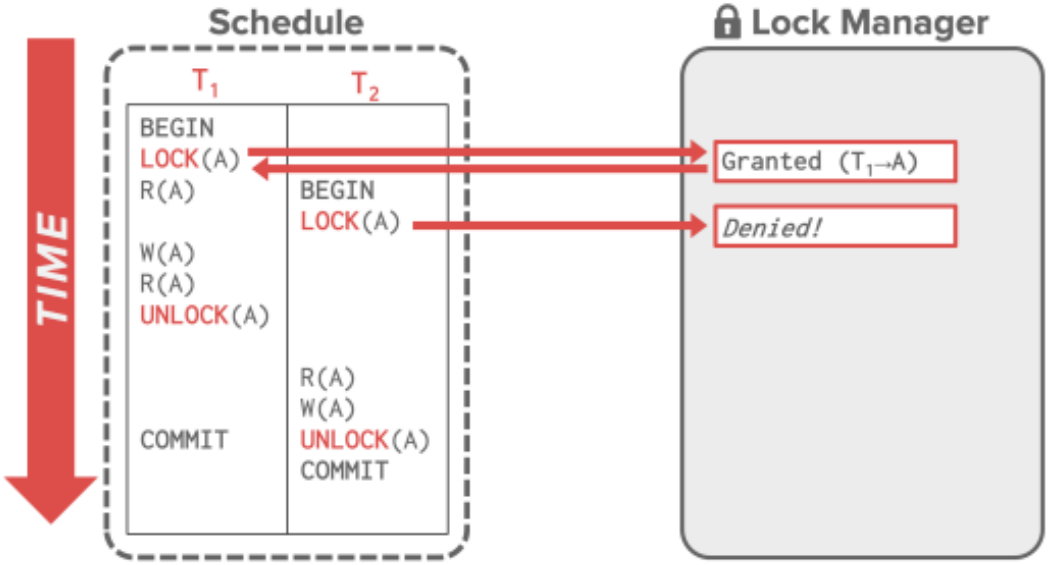
  T_2: **lock-S**(*A*);
  read (*A*);
  unlock(*A*);
  lock-S(*B*);
  read (*B*);
  unlock(*B*);
  display(*A*+*B*)

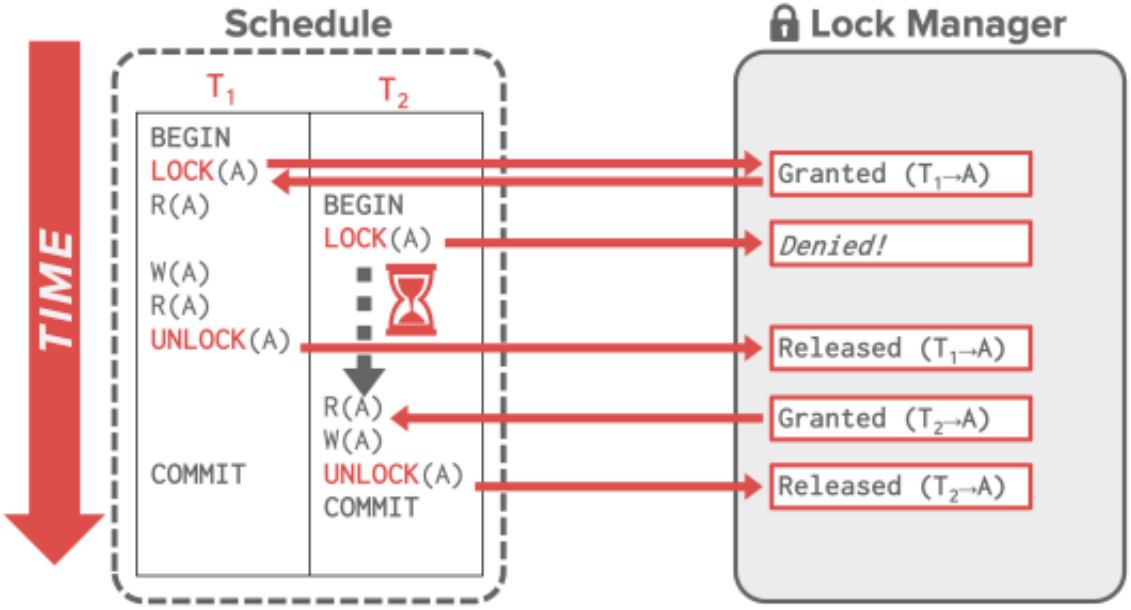
- Locking as above is not sufficient to guarantee serializability — if *A* and *B* get updated in-between the read of *A* and *B*, the displayed sum would be wrong.
- A **locking protocol** is a set of rules followed by all transactions while requesting and releasing locks. Locking protocols restrict the set of possible schedules.

©Silberschatz, Korth and Sudarshan



Executing with Locks





 ©Silberschatz, Korth and Sudarshan



The Two-Phase Locking Protocol (2PC)

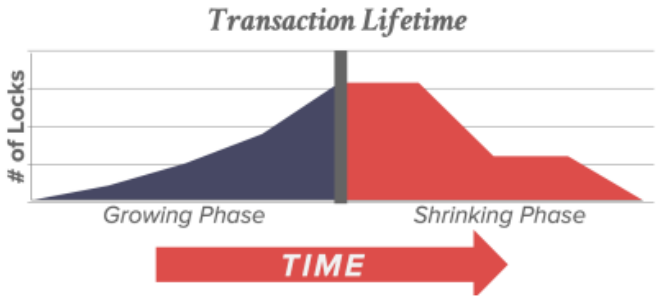
- This protocol ensures conflict-serializable schedules.
- Phase 1: Growing Phase
  - Transaction may obtain locks
  - Transaction may not release locks
- Phase 2: Shrinking Phase
  - Transaction may release locks
  - Transaction may not obtain locks
- The protocol assures serializability. It can be proved that the transactions can be serialized in the order of their **lock points** (i.e., the point where a transaction acquired its final lock).

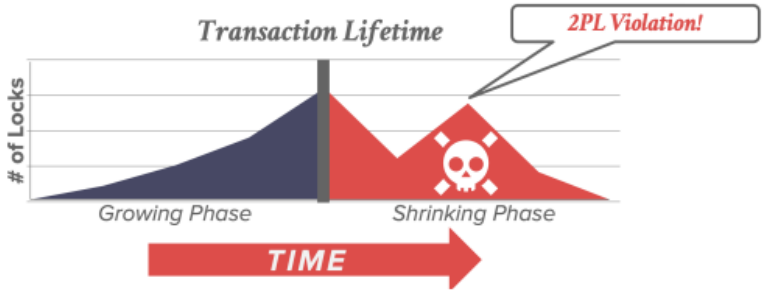
©Silberschatz, Korth and Sudarshan



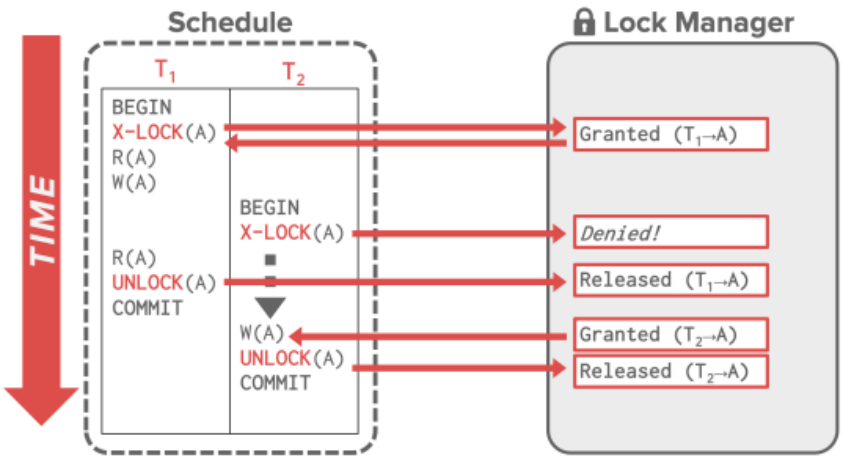
The Two-Phase Locking Protocol (2PC)

- The transaction is not allowed to acquire/upgrade locks after the growing phase finishes.





Executing with 2PL





Lock Conversions

- Two-phase locking with lock conversions:
  - First Phase:
    - can acquire a lock-S on item
    - can acquire a lock-X on item
    - can convert a lock-S to a lock-X (upgrade)
  - Second Phase:
    - can release a lock-S
    - can release a lock-X
    - can convert a lock-X to a lock-S (downgrade)
- This protocol assures serializability. But still relies on the programmer to insert the various locking instructions.

©Silberschatz, Korth and Sudarshan



Automatic Acquisition of Locks

- A transaction T_i issues the standard read/write instruction, without explicit locking calls.
- The operation **read**(D) is processed as:

```
if Ti has a lock on D
  then
        read(D)
  else
     begin
          if necessary wait until no other
             transaction has a lock-X on D
          grant Ti a  lock-S on D;
          read(D)
        end
```


©Silberschatz, Korth and Sudarshan



Automatic Acquisition of Locks (Cont.)

- write*(D)* is processed as:

```
if Ti has a  lock-X on D
   then
     write(D)
  else begin
       if necessary wait until no other transaction has any lock on D,
       if Ti has a lock-S on D
            then
                upgrade lock on D  to lock-X
           else
                grant Ti a lock-X on D
           write(D)
    end;
```

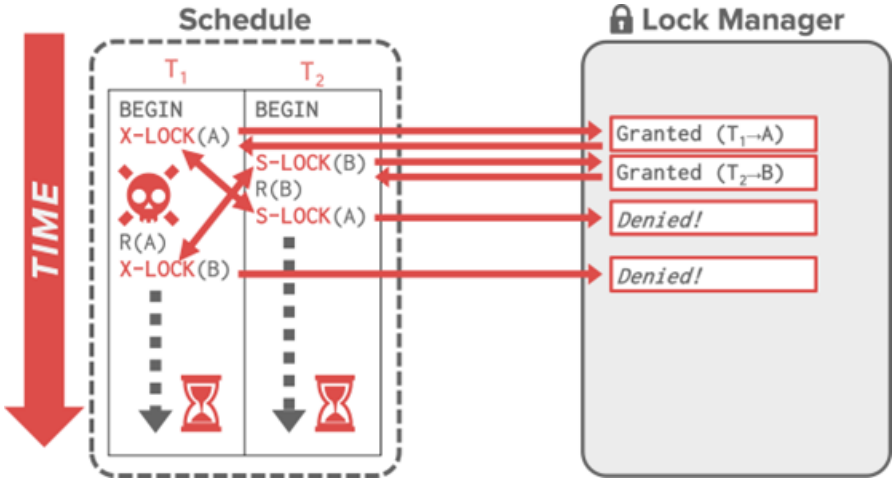
- All locks are released after commit or abort



Deadlocks

- Consider the partial schedule

T_3	T_4
lock-x (B)	
read (B)	
$B := B - 50$	
write (B)	
	lock-s (A)
	read (A)
	lock-s (B)
lock-x (A)	



- Neither T_3 nor T_4 can make progress — executing **lock-S**(*B*) causes T_4 to wait for T_3 to release its lock on *B*, while executing **lock-X**(*A*) causes T_3 to wait for T_4 to release its lock on *A*.
- Such a situation is called a **deadlock**.
  - To handle a deadlock one of T_3 or T_4 must be rolled back and its locks released.



Deadlocks (Cont.)

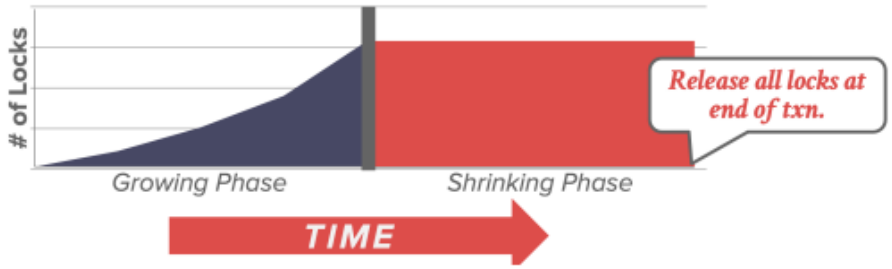
- Two-phase locking *does not* ensure freedom from deadlocks.
- In addition to deadlocks, there is a possibility of **starvation.**
- **Starvation** occurs if the concurrency control manager is badly designed. For example:
  - A transaction may be waiting for an X-lock on an item, while a sequence of other transactions request and are granted an S-lock on the same item.
  - The same transaction is repeatedly rolled back due to deadlocks.
- Concurrency control manager can be designed to prevent starvation.

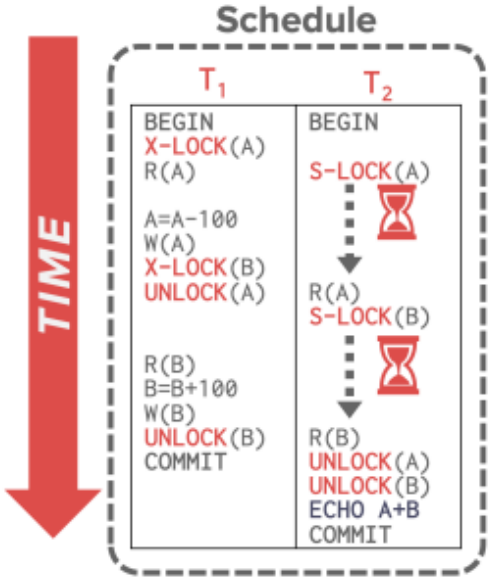
©Silberschatz, Korth and Sudarshan



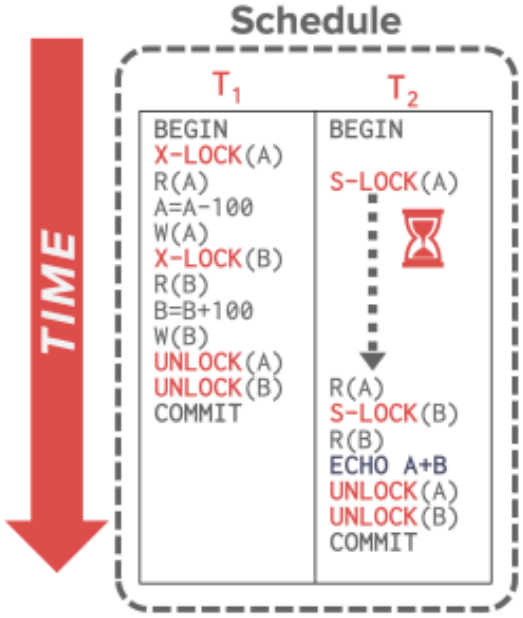
Deadlocks (Cont.)

- The potential for deadlock exists in most locking protocols. Deadlocks are a necessary evil.
- When a deadlock occurs there is a possibility of cascading roll-backs.
- Cascading roll-back is possible under two-phase locking. To avoid this, follow a modified protocol called **strict two-phase locking** -- a transaction must hold all its exclusive locks till it commits/aborts.
- **Rigorous two-phase locking** is even stricter. Here, *all* locks are held till commit/abort. In this protocol transactions can be serialized in the order in which they commit.





Strong Strict 2PL Example



©Silberschatz, Korth and Sudarshan



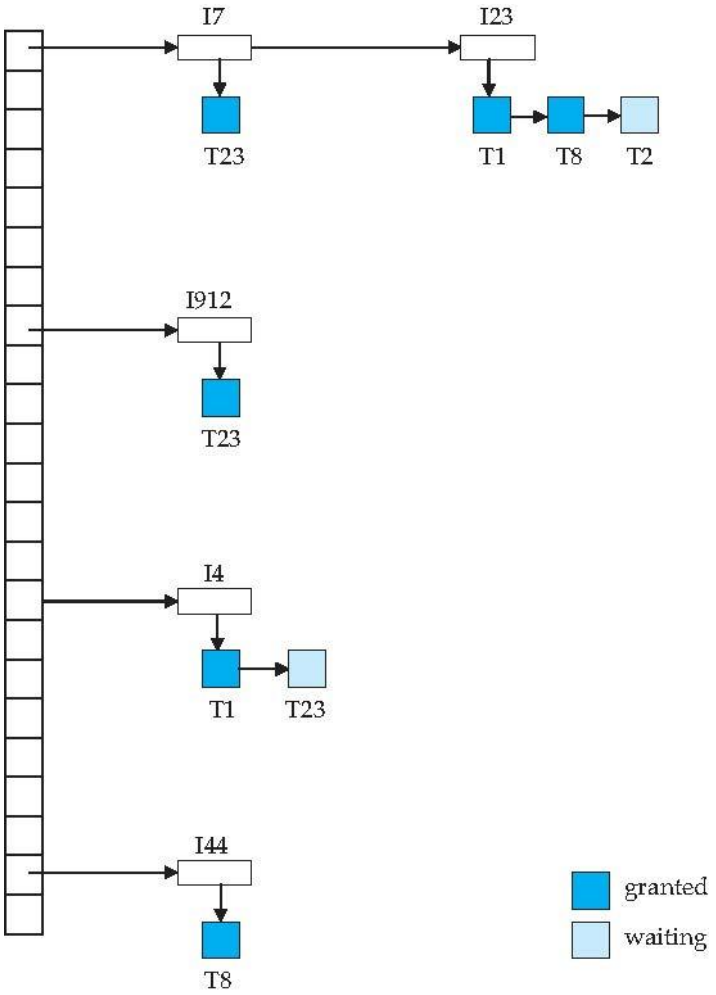
Implementation of Locking

- A **lock manager** can be implemented as a separate process to which transactions send lock and unlock requests
- The lock manager replies to a lock request by sending a lock grant messages (or a message asking the transaction to roll back, in case of a deadlock)
- The requesting transaction waits until its request is answered
- The lock manager maintains a data-structure called a **lock table** to record granted locks and pending requests
- The lock table is usually implemented as an in-memory hash table indexed on the name of the data item being locked

©Silberschatz, Korth and Sudarshan



Lock Table



- Dark blue rectangles indicate granted locks; light blue indicate waiting requests
- Lock table also records the type of lock granted or requested
- New request is added to the end of the queue of requests for the data item, and granted if it is compatible with all earlier locks
- Unlock requests result in the request being deleted, and later requests are checked to see if they can now be granted
- If transaction aborts, all waiting or granted requests of the transaction are deleted
  - lock manager may keep a list of locks held by each transaction, to implement this efficiently



IMPORTANT TERMS

1) **DIRTY READ :** Read(A) is called dirty read, when it reads the value Data item A which is uncommitted.

2) **STRICT SCHEDULE :** A schedule in which a transaction cannot **read or write a data item until the transaction that last wrote it commits or aborts**, thereby preventing cascading rollbacks.

Transaction T1	Transaction T2
R (A)	
W (A)	
⋮	R (A) // Dirty Read
	W (A)
	Commit
Failure	

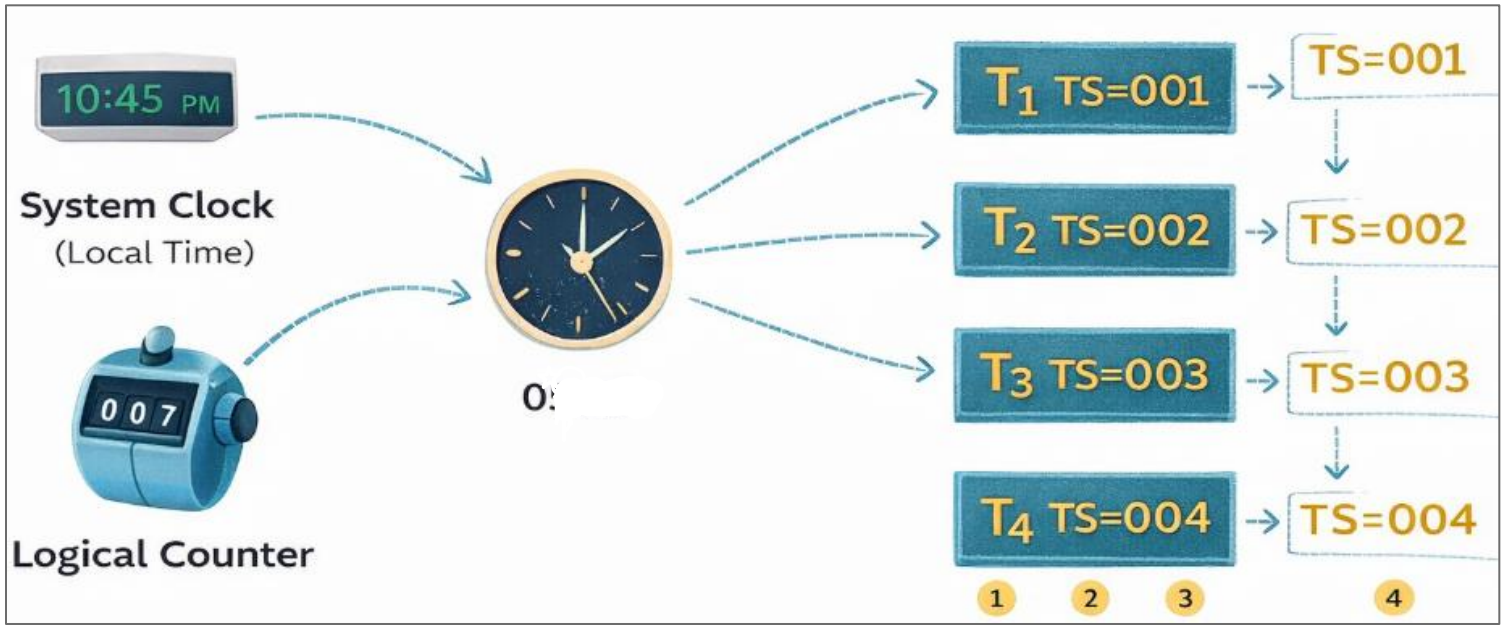
T1	T2
R (A)	
W (A)	
	W (A) // Uncommitted Write
Commit	

Transaction T1	Transaction T2
R (A)	
W (B)	
	R (A)
R (B)	
W (B)	
Commit	
	R (B)
	Commit



CONCEPT OF TIMESTAMPING

Every process that enters the queue is assigned a unique Timestamp(Maybe a local time or a logical counter). Same as ticket counter we see in any restaurant.



The larger the Timestamp the younger transaction. Transactions are executed in the order of their timestamps.



Timestamp-Based Protocols

- Each transaction is issued a timestamp when it enters the system. If an old transaction T_i has time-stamp TS(T_i), a new transaction T_j is assigned time-stamp TS(T_j) such that TS(T_i) <TS(T_j).
- The protocol manages concurrent execution such that the time-stamps determine the serializability order.
- Multiple implementation strategies:
  - Physical system clock (*e.g.*, timezones)
  - Logical counter (*e.g.*, overflow)
  - Hybrid
- In order to assure such behavior, the protocol maintains for each data Q two timestamp values:
  - **W-timestamp**(Q) is the largest time-stamp of any transaction that executed **write**(Q) successfully.
  - **R-timestamp**(Q) is the largest time-stamp of any transaction that executed **read**(Q) successfully.

©Silberschatz, Korth and Sudarshan



Timestamp-Based Protocols (Cont.)

- The timestamp ordering protocol ensures that any conflicting **read** and **write** operations are executed in timestamp order.
- Suppose a transaction T_i issues a **read**(Q)
  1. If TS(T_i) ≤ **W**-timestamp(Q), then T_i needs to read a value of Q that was already overwritten.
     - Hence, the **read** operation is rejected, and T_i is rolled back.
  2. If TS(T_i) ≥ **W**-timestamp(Q), then the **read** operation is executed, and R-timestamp(Q) is set to **max**(R-timestamp(Q), TS(T_i)).

Basic T/O – Reads

- If $TS(T_i) < W - TS(X)$, this violates timestamp order of T_i with regard to the writer of X.
  - Abort T_i and restart it with a newer TS (so that is later than the writer of X).
- Else:
  - Allow T_i to read X.
  - Update $R - TS(X)$ to $\max(R - TS(X), TS(T_i))$
  - Have to make a local copy of X to ensure repeatable reads for T_i.

©Silberschatz, Korth and Sudarshan



Timestamp-Based Protocols (Cont.)

Suppose that transaction T_i issues **write**(Q).

1. If TS(T_i) < R-timestamp(Q), then the value of Q that T_i is producing was needed previously, and the system assumed that that value would never be produced.
   - Hence, the **write** operation is rejected, and T_i is rolled back.
2. If TS(T_i) < W-timestamp(Q), then T_i is attempting to write an obsolete value of Q.
   - Hence, this **write** operation is rejected, and T_i is rolled back.
3. Otherwise, the **write** operation is executed, and W-timestamp(Q) is set to TS(T_i).

Basic T/O – Writes

- If $TS(T_i) < R - TS(X)$ or $TS(T_i) < W - TS(X)$
  - Abort and restart T_i.
- Else:
  - Allow T_i to write X and update $W - TS(X)$
  - Also have to make a local copy of X to ensure repeatable reads for T_i.

©Silberschatz, Korth and Sudarshan



Timestamp Protocol **coding**

Transaction T issues read(X)

```
IF (W_TS(X) > TS(T)) THEN
        abort T
        rollback T
ELSE
        perform read(X)
        R_TS(X) = max(R_TS(X), TS(T))
```

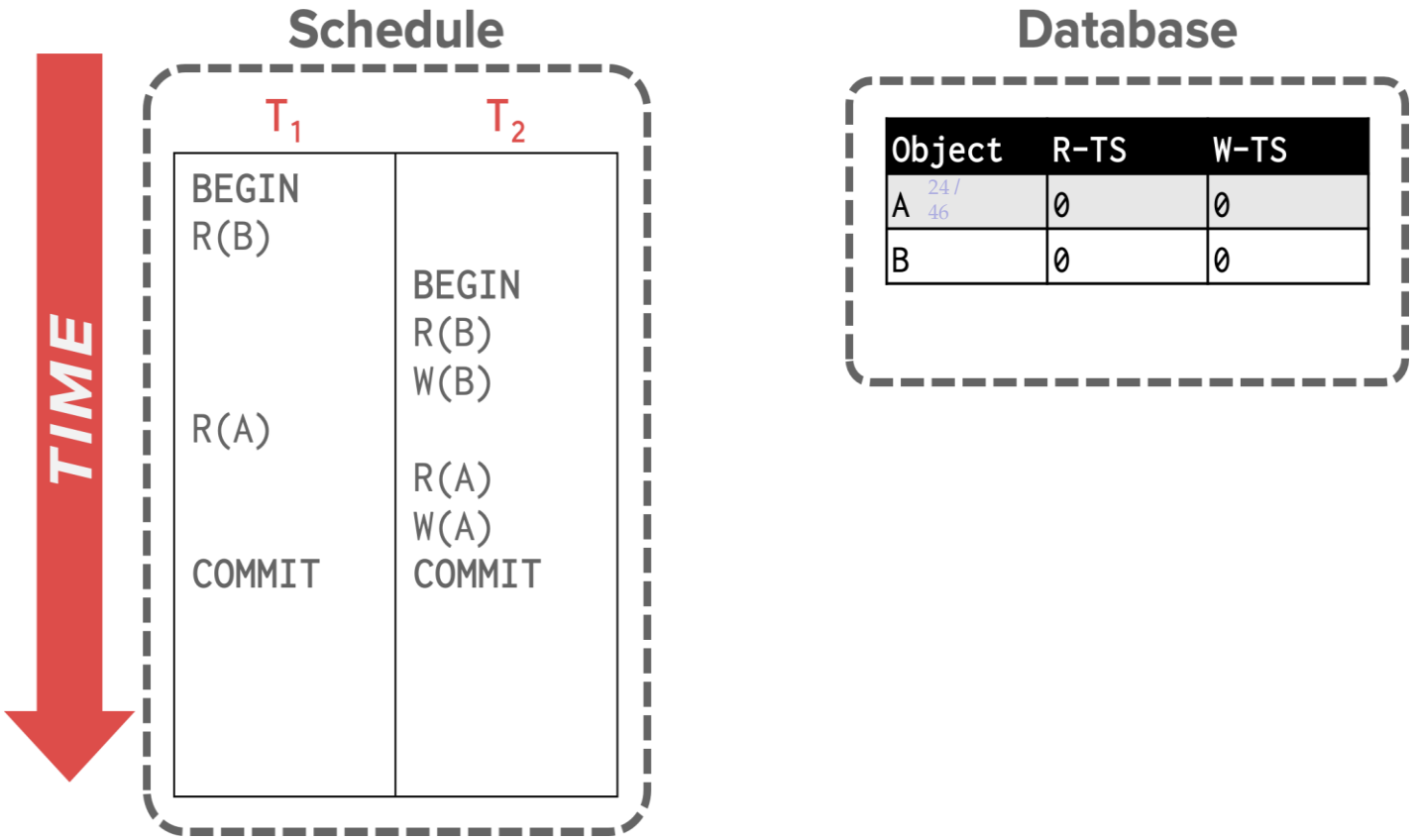
Transaction T issues write(X)

```
IF (R_TS(X) > TS(T)) THEN
        abort T
        rollback T
ELSE IF (W_TS(X) > TS(T)) THEN
        abort T
        rollback T
ELSE
        perform write(X)
        W_TS(X) = TS(T)
```


©Silberschatz, Korth and Sudarshan

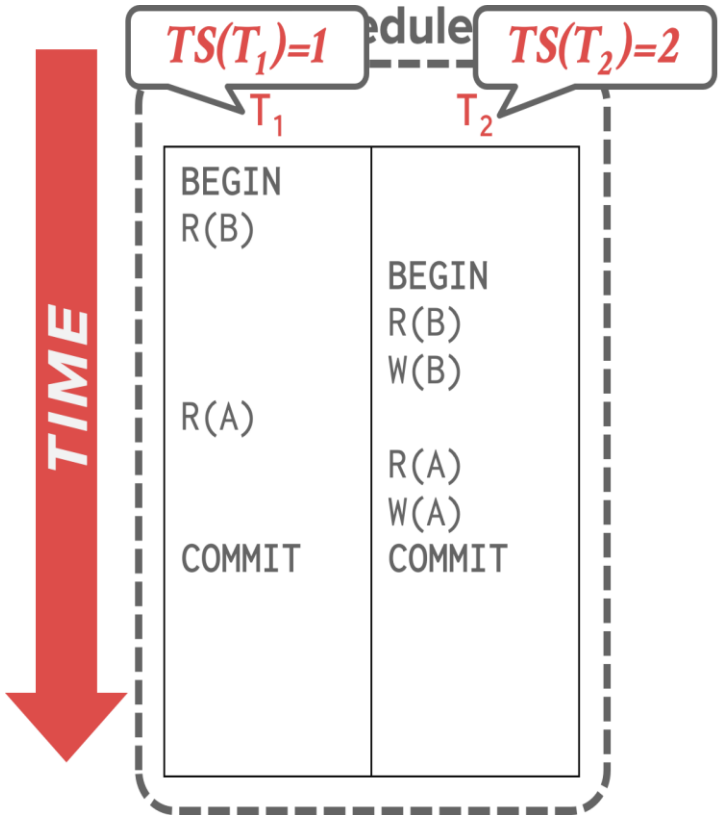


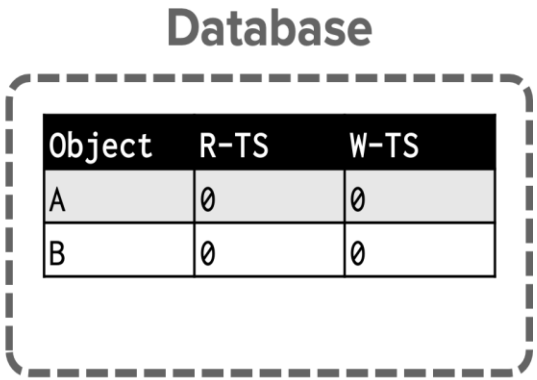
Basic T/O – Example 1





Basic T/O – Example 1

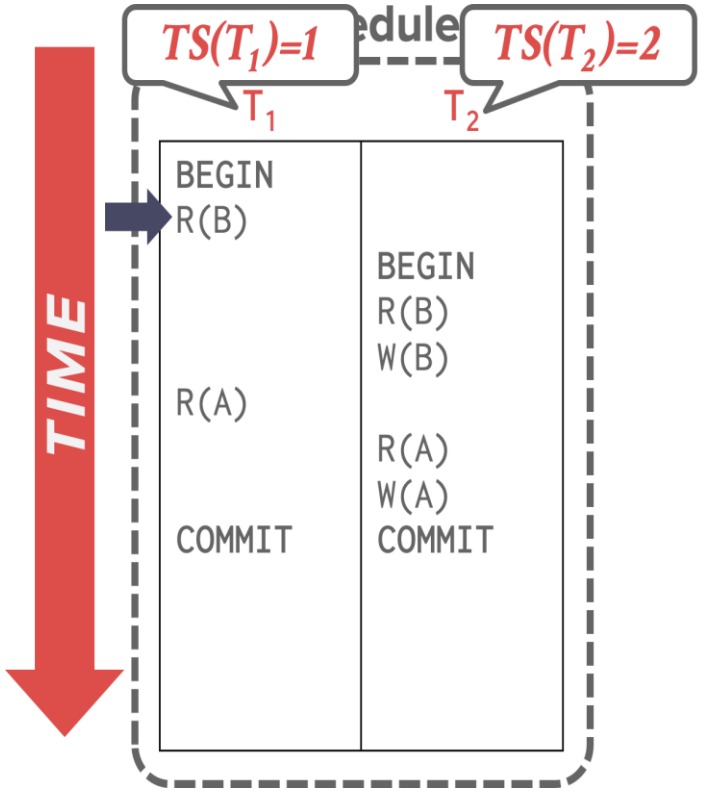


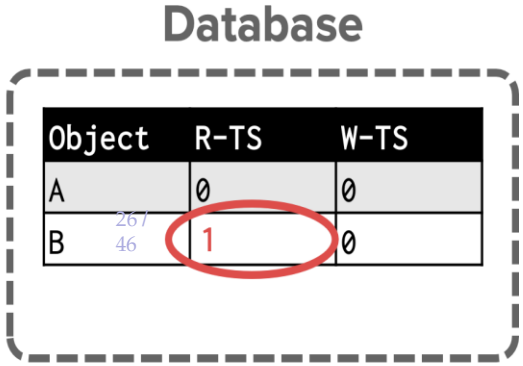


 ©Silberschatz, Korth and Sudarshan



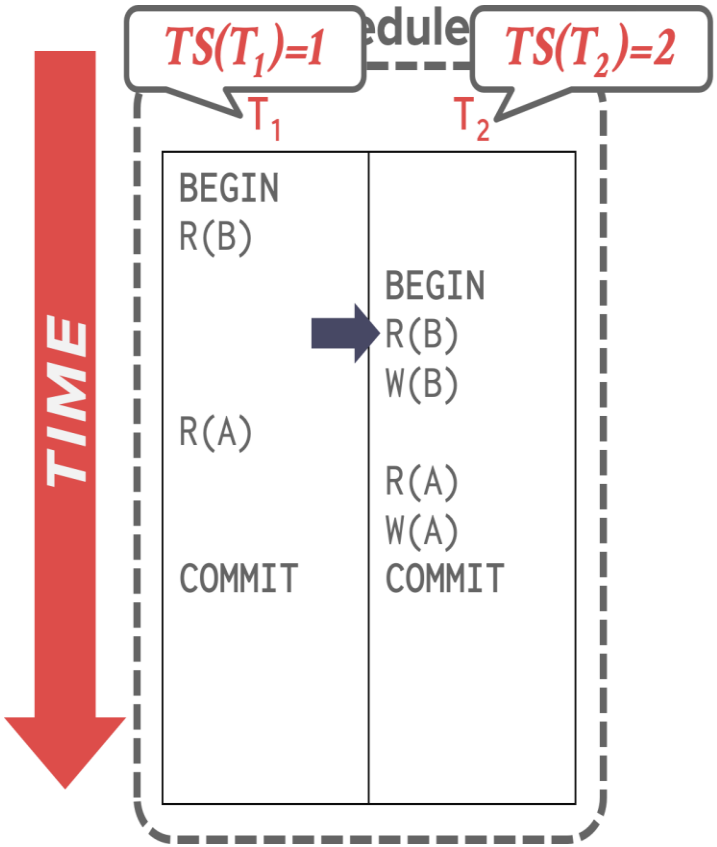
Basic T/O – Example 1







Basic T/O – Example 1

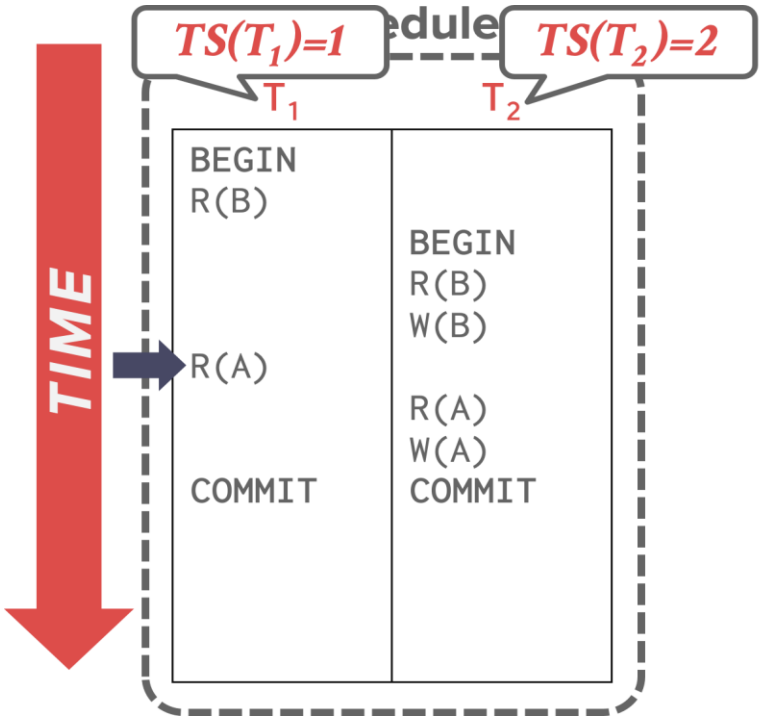


Database

Object	R-TS	W-TS
A	0	0
B	2	0



Basic T/O – Example 1

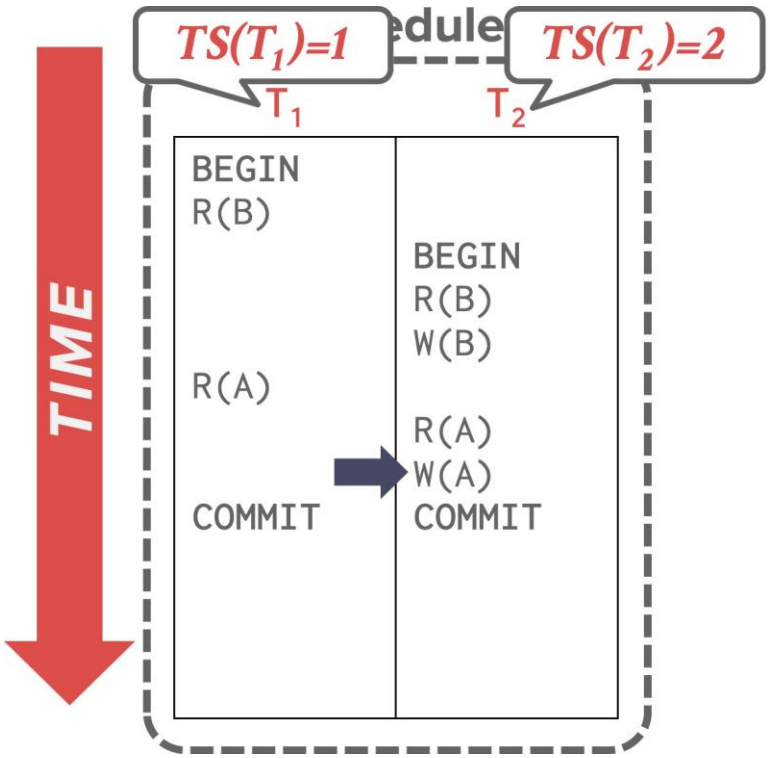


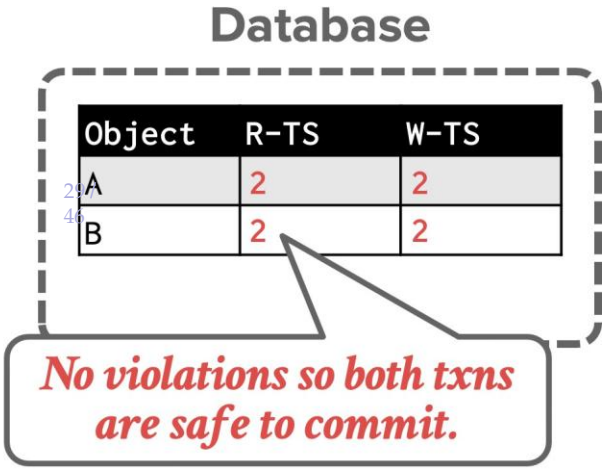
Database

Object	R-TS	W-TS
A	1	0
B	2	2



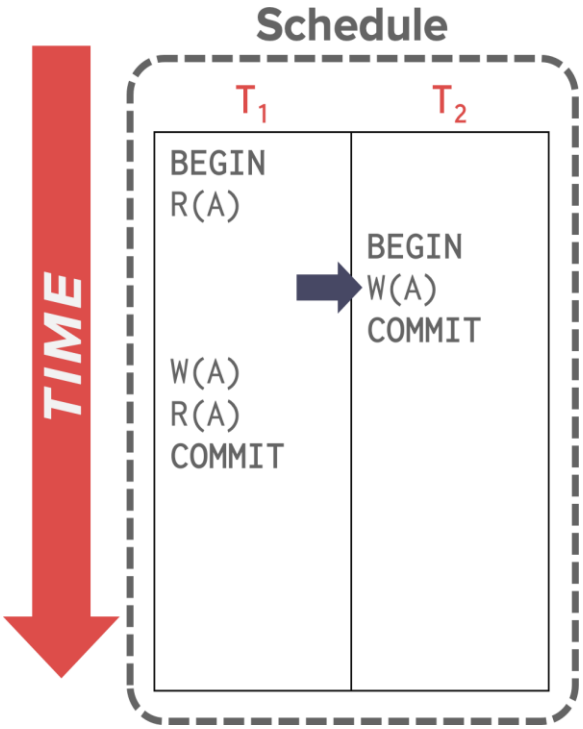
Basic T/O – Example 1







Basic T/O – Example 1

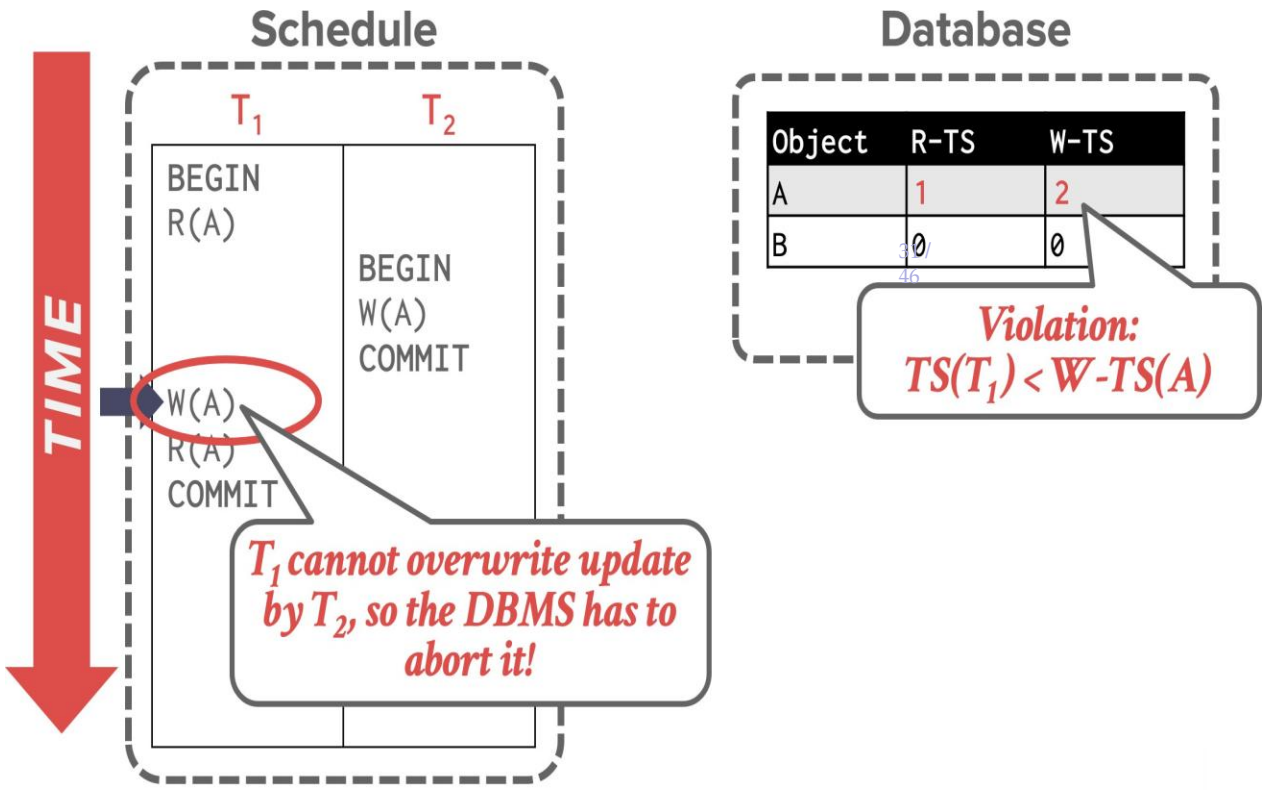


Database

Object	R-TS	W-TS
A	1	2
B	0	0



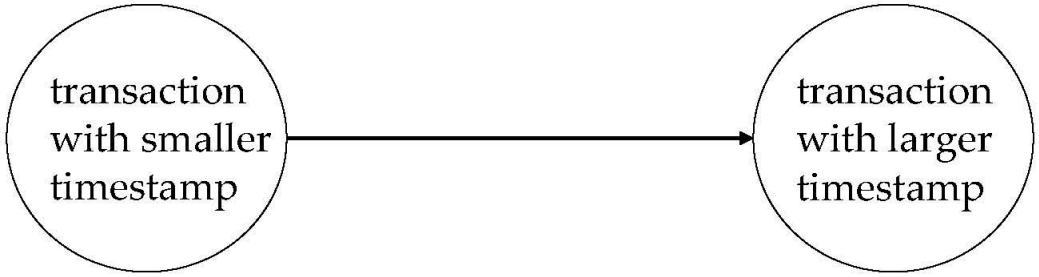
Basic T/O – Example 1





Correctness of Timestamp-Ordering Protocol

- The timestamp-ordering protocol guarantees serializability since all the arcs in the precedence graph are of the form:



Thus, there will be no cycles in the precedence graph

- Timestamp protocol ensures freedom from deadlock as no transaction ever waits.
- But the schedule may not be cascade-free, and may not even be recoverable.



Recoverability and Cascade Freedom

- **Problem with timestamp-ordering protocol:**
  - Suppose T_i aborts, but T_j has read a data item written by T_i
  - Then T_j must abort; if T_j had been allowed to commit earlier, the schedule is not recoverable.
  - Further, any transaction that has read a data item written by T_j must abort
  - This can lead to cascading rollback --- that is, a chain of rollbacks
- Solution 1:
  - A transaction is structured such that its writes are all performed at the end of its processing
  - All writes of a transaction form an atomic action; no transaction may execute while a transaction is being written
  - A transaction that aborts is restarted with a new timestamp
- Solution 2:
  - Limited form of locking: wait for data to be committed before reading it
- Solution 3:
  - Use commit dependencies to ensure recoverability

©Silberschatz, Korth and Sudarshan



Deadlocks



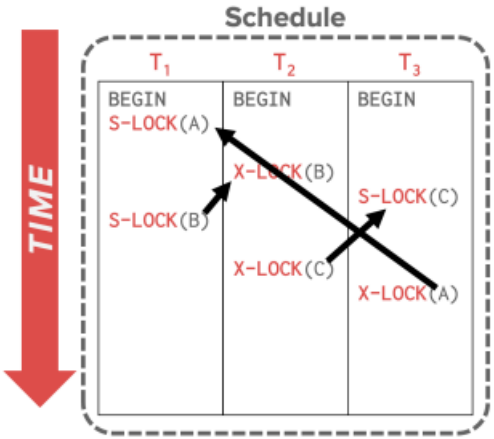
- Consider the following two transactions:

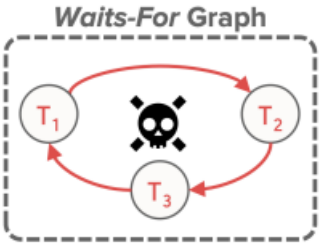
  T_1: write (X) T_2: write(Y)

  write(Y) write(X)

- Schedule with deadlock

T_1	T_2
lock-X on A	
write (A)	
	lock-X on B
	write (B)
	wait for **lock-X** on A
wait for **lock-X** on B	







Deadlock Handling

- System is deadlocked if there is a set of transactions such that every transaction in the set is waiting for another transaction in the set.
- ***Deadlock prevention*** protocols ensure that the system will *never* enter into a deadlock state. Some prevention strategies :
  - Require that each transaction locks all its data items before it begins execution (predeclaration).
  - Impose partial ordering of all data items and require that a transaction can lock data items only in the order specified by the partial order.



More Deadlock Prevention Strategies

- Following schemes use transaction timestamps for the sake of deadlock prevention alone.
- **wait-die** scheme — non-preemptive
  - older transaction may wait for younger one to release data item. (older means smaller timestamp) Younger transactions never Younger transactions never wait for older ones; they are rolled back instead.
  - a transaction may die several times before acquiring needed data item
- **wound-wait** scheme — preemptive
  - older transaction *wounds* (forces rollback) of younger transaction instead of waiting for it. Younger transactions may wait for older ones.
  - may be fewer rollbacks than *wait-die* scheme.

In otherwards, If a transaction ***Ti*** requests a lock and a transaction ***Tj*** holds a conflicting lock, the lock manager can use one of the following policies:

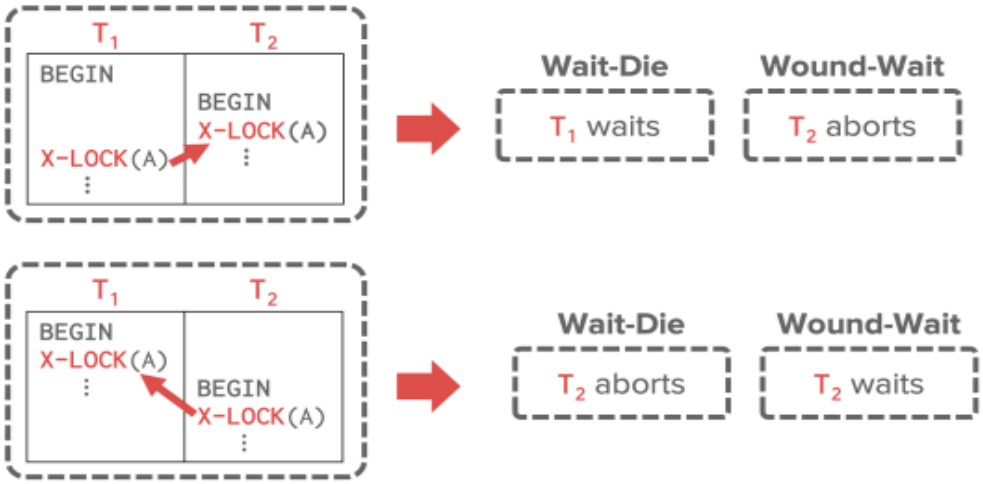
- Wound-Wait: If ***Ti*** has higher priority, ***Tj*** is aborted; otherwise, ***Ti*** waits
- Wait-Die: If ***Ti*** has higher priority, it is allowed to wait; otherwise, it is aborted

©Silberschatz, Korth and Sudarshan



Timestamps: for the sake of deadlock prevention

- Assign priorities based on timestamps:
  - Older Timestamp = Higher Priority (*e.g.*, T1 > T2)
- **Wait-Die** ("Old Waits for Young")
  - If requesting txn has higher priority than holding txn, then requesting txn waits for holding txn.
  - Otherwise requesting txn aborts.
- **Wound-Wait** ("Young Waits for Old")
  - If requesting txn has higher priority than holding txn, then holding txn aborts and releases lock.
  - Otherwise requesting txn waits.



 ©Silberschatz, Korth and Sudarshan



Deadlock prevention (Cont.)

- Both in *wait-die* and in *wound-wait* schemes, a rolled back transactions is restarted with its original timestamp. Older transactions thus have precedence over newer ones, and starvation is hence avoided.
- **Timeout-Based Schemes:**
  - a transaction waits for a lock only for a specified amount of time. If the lock has not been granted within that time, the transaction is rolled back and restarted,
  - Thus, deadlocks are not possible
  - simple to implement; but starvation is possible. Also difficult to determine good value of the timeout interval.



Deadlock Detection

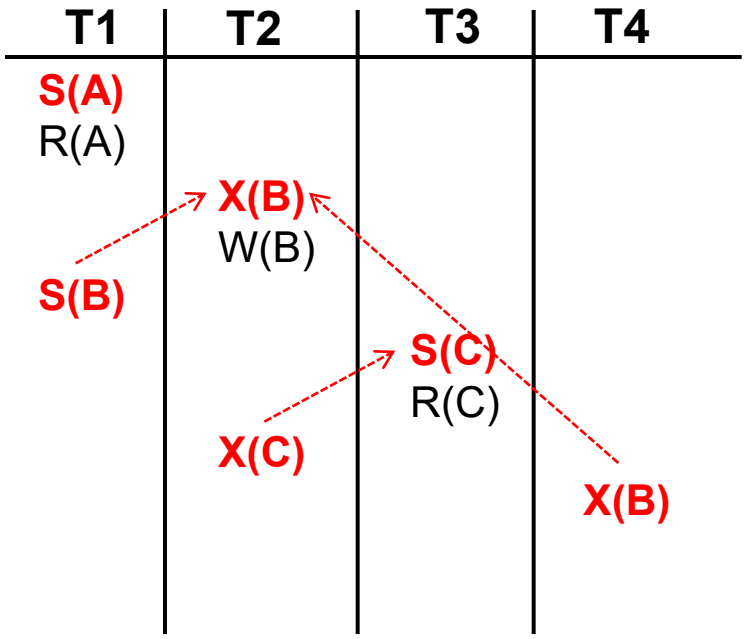
- Deadlocks can be described as a *wait-for graph*, which consists of a pair $G = (V,E)$,
  - V is a set of vertices (all the transactions in the system)
  - E is a set of edges; each element is an ordered pair $T_i \rightarrow T_j$.
- If $T_i \rightarrow T_j$ is in E, then there is a directed edge from T_i to T_j, implying that T_i is waiting for T_j to release a data item.
- When T_i requests a data item currently being held by T_j, then the edge $T_i \rightarrow T_j$ is inserted in the wait-for graph. This edge is removed only when T_j is no longer holding a data item needed by T_i.
- The system is in a deadlock state if and only if the wait-for graph has a cycle. Must invoke a deadlock-detection algorithm periodically to look for cycles.

©Silberschatz, Korth and Sudarshan

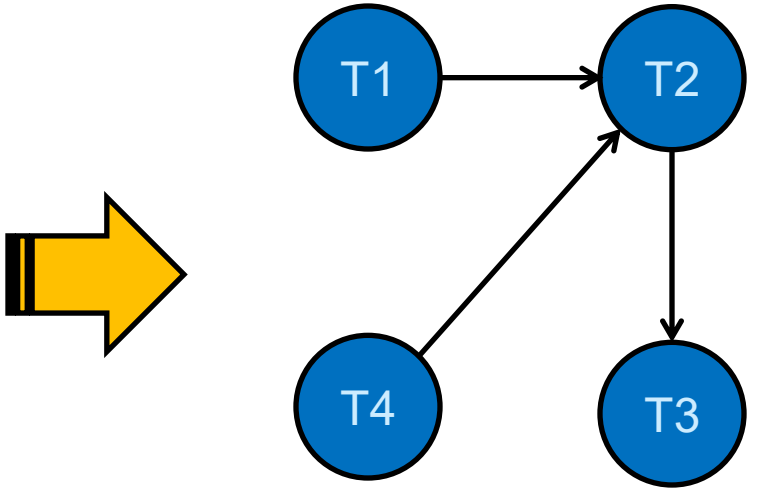


Deadlock Detection (*Cont'd*)

- The following schedule is free of deadlocks:



A schedule *without* a deadlock



No cycles; hence, no deadlocks!

The Corresponding **Waits-For Graph***

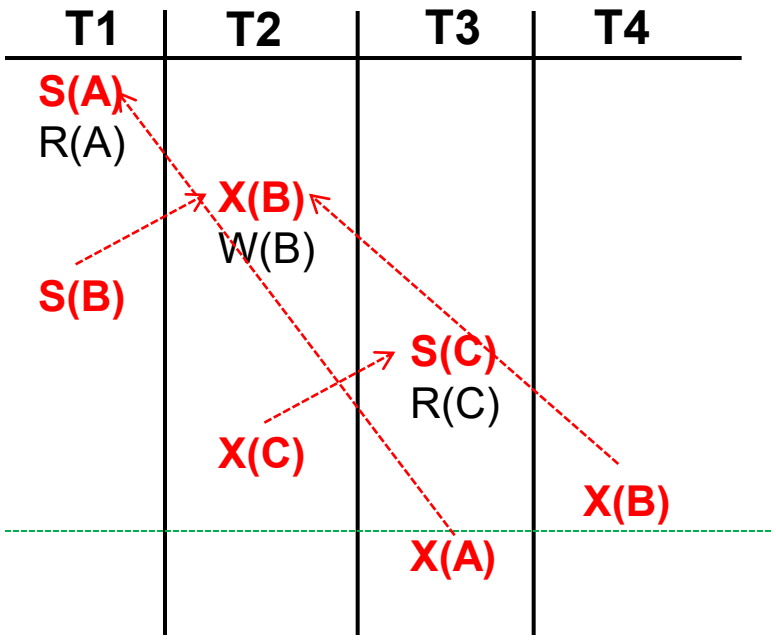
*The nodes correspond to active transactions and there is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

©Silberschatz, Korth and Sudarshan

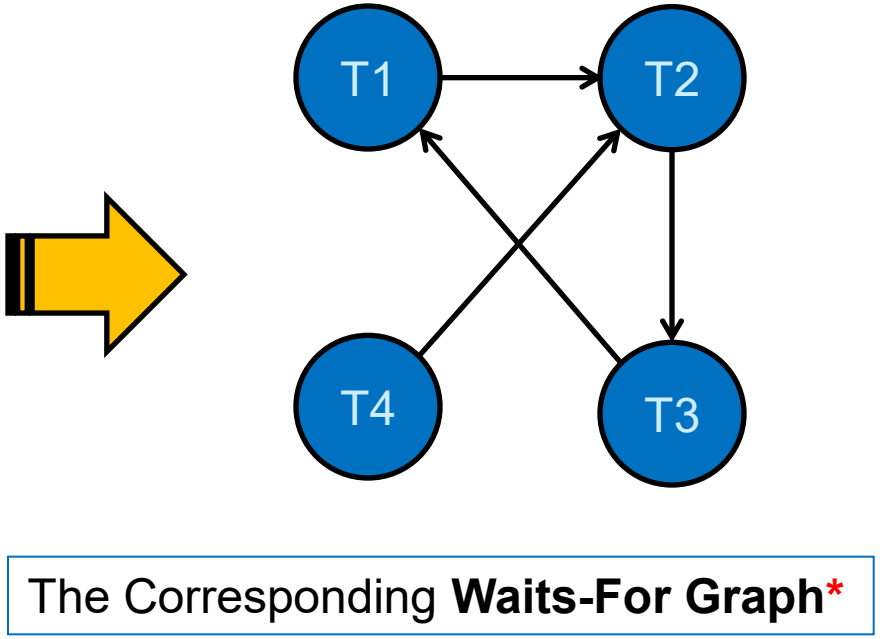


Deadlock Detection (*Cont'd*)

- The following schedule is **NOT** free of deadlocks:



A schedule *with* a deadlock



The Corresponding **Waits-For Graph***

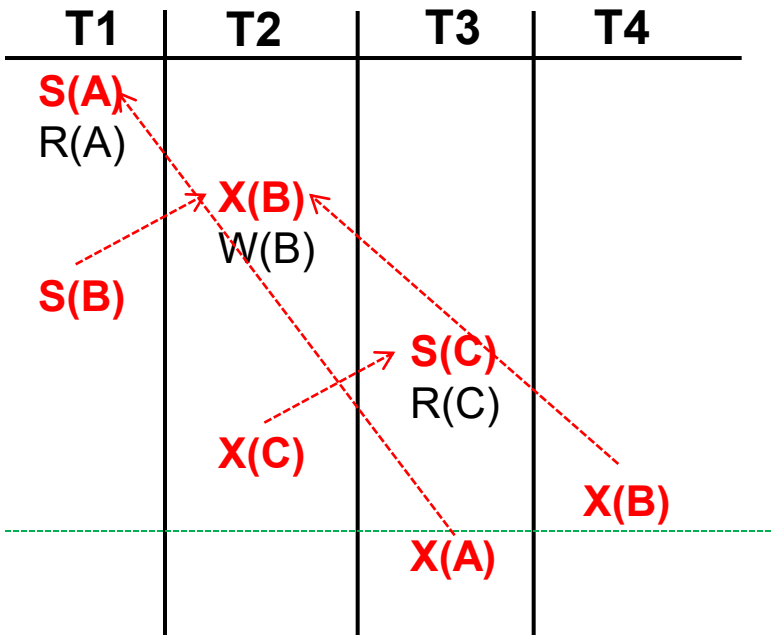
*The nodes correspond to active transactions and there is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

©Silberschatz, Korth and Sudarshan

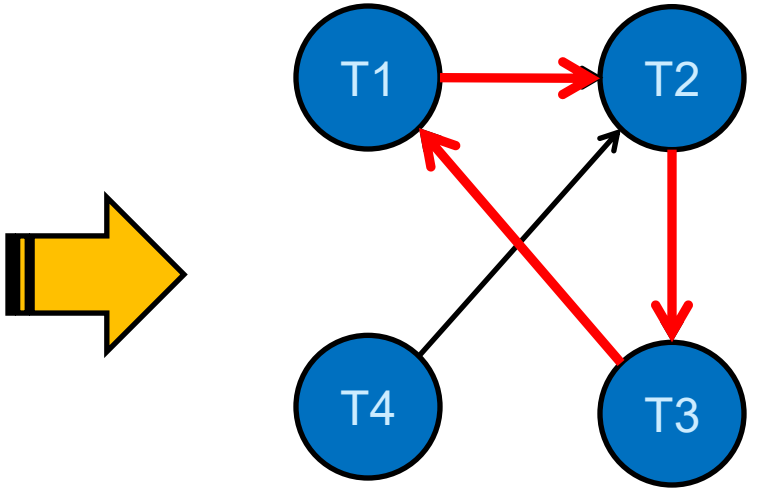


Deadlock Detection (*Cont'd*)

- The following schedule is **NOT** free of deadlocks:



A schedule *with* a deadlock



Cycle detected; hence, a deadlock!

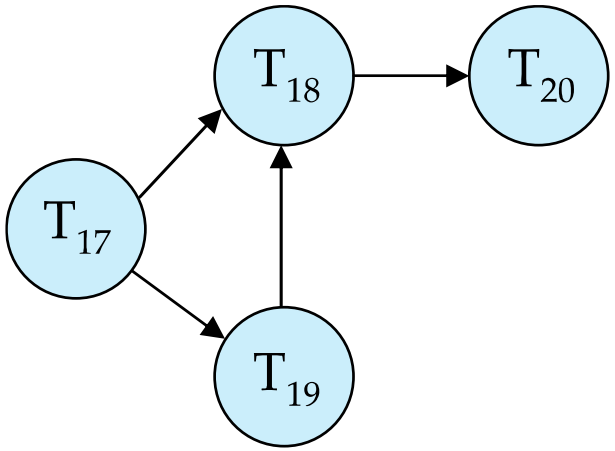
The Corresponding **Waits-For Graph***

*The nodes correspond to active transactions and there is an edge from Ti to Tj *if and only if* Ti is waiting for Tj to release a lock

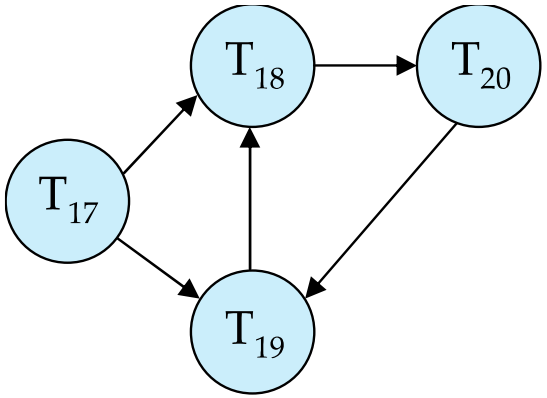
©Silberschatz, Korth and Sudarshan



Deadlock Detection (Cont.)



Wait-for graph without a cycle



Wait-for graph with a cycle

©Silberschatz, Korth and Sudarshan



Deadlock Recovery

- When deadlock is detected :
  - Some transaction will have to rolled back (made a victim) to break deadlock. Select that transaction as victim that will incur minimum cost.
  - Rollback -- determine how far to roll back transaction
    - Total rollback: Abort the transaction and then restart it.
    - More effective to roll back transaction only as far as necessary to break deadlock.
  - Starvation happens if same transaction is always chosen as victim. Include the number of rollbacks in the cost factor to avoid starvation

©Silberschatz, Korth and Sudarshan




Thank
You